Introduction to System Software

System Programming

> System Programming is the act of building Systems Software using System Programming Languages. In the computer hierarchy, the order is:

1. Hardware
2. System Programs
3. Application Programs

System software serves as the interface between the hardware and the end users.

Types of Software

There are two main types of software:

- Systems software: Programs dedicated to managing the computer itself.
- Application software: Programs that control and manage the operations of computer hardware and help application programs to execute correctly.

System Programs

Various system programs include:

- Assemblers
- Loaders
- Linkers
- Macro processors
- Compilers
- Interpreters
- Operating Systems
- Device drivers

Macro Processors or Preprocessors

> Preprocessors are programs that process our source code before compilation.

There are four main types of pre-processor directives:

- Macros
- File Inclusion
- Conditional Compilation
- Other directives

Macros

> Macros are a piece of code in a program which is given some name. Whenever this name is encountered by the compiler, the compiler replaces the name with the actual piece of code.

- The `#define` directive is used to define a macro.
- Macro definitions need not be terminated by a semi-colon (;).

```c
#include <stdio.h>

// Macro definition
#define LIMIT 5

int main() {
    // Print the value of macro defined
    printf("The value of LIMIT is %d", LIMIT);
    return 0;
}
```

Output:

```
The value of LIMIT is 5
```

File Inclusion

This preprocessor directive tells the compiler to include a file in the source code program. There are two types of files which can be included:

- Header Files or Standard files
- User-defined files

Header Files or Standard Files

> These files contain definitions of pre-defined functions like `printf()`, `scanf()`, etc.

- These files must be included for working with these functions.
- Different functions are declared in different header files.
  - C++ standard I/O functions are in the `iostream` header file.
  - Functions that perform string operations are in the `string` header file.

Syntax: `#include< file_name >`

The `<` and `>` brackets tell the compiler to look for the file in the standard directory.

User Defined Files

When a program becomes very large, it is good practice to divide it into smaller files and include them whenever needed.

Syntax: `#include"filename"`

Syntax	Description
`#include<filename.h>`	Loaded from the default directory (e.g., `C:\TC\INCLUDE`) for pre-defined header files.
`#include "filename.h"`	Loaded from the current working directory for user-defined header files.

Conditional Compilation Directives

> These directives help to compile a specific portion of the program or to skip compilation of some specific part of the program based on certain conditions.

- This can be done with the help of preprocessing commands `ifdef` and `endif`.

Syntax:

```
#ifdef macro_name
    statement1;
    statement2;
    statement3;
    ...
    statementN;
#endif
```

If the macro with the specified `macro_name` is defined, then the block of statements will execute normally; otherwise, the compiler will skip this block of statements.

```
#include <stdio.h>

#define YEARS_OLD 10

int main() {
    #ifdef YEARS_OLD
        printf("TechOnTheNet is over %d years old.\n", YEARS_OLD);
    #endif
    printf("TechOnTheNet is a great resource.\n");
    return 0;
}
```

The `#ifdef` directive allows for conditional compilation. The preprocessor determines if the provided macro exists before including the subsequent code in the compilation process.

Other Directives

Apart from the above directives, there are two more directives which are not commonly used:

- `#undef` Directive
- `#pragma` Directive

`#undef` Directive

The `#undef` directive is used to undefine an existing macro.

```
#undef LIMIT
```

Using this statement will undefine the existing macro `LIMIT`. After this statement, every `#ifdef LIMIT` statement will evaluate to false.

`#pragma` Directive

> This directive is a special-purpose directive and is used to turn on or off some features.

- These directives are compiler-specific, i.e., they vary from compiler to compiler.

Program Execution with C Compiler

1. The user writes a program in C (a high-level language).
2. The C compiler compiles the program and translates it to an assembly program.
3. An assembler then translates the assembly program into machine code (object file).
4. A linker tool is used to link all the parts of the program together for execution (executable machine code).
5. A loader loads all of them into memory, and then the program is executed.

Compilers

> A compiler is a software that translates the code written in one language to some other language without changing the meaning of the program.

- The compiler also makes the target code efficient and optimized in terms of time and space.
- In this example, the C compiler converts a high-level language program into assembly language.
- The language processor that reads the complete source program written in a high-level language as a whole in one go and translates it into an equivalent program in machine language is called a **Compiler**.
- In a compiler, the source code is translated to object code successfully if it is free of errors.
- The compiler specifies the errors at the end of compilation with line numbers when there are any errors in the source code. The errors must be removed before the compiler can successfully recompile the source code again.

Assemblers

A program that converts **assembly language programs** into **object files**.

- The output of an assembler is called an object file.
- Object files contain a combination of machine instructions, data, and information needed to place instructions properly in memory.

Object File Format

- **Object file header**: Describes the size and position of the other pieces of the file.
- **Text segment**: Contains the machine instructions.
- **Data segment**: Contains the binary representation of data in the assembly file.
- **Relocation info**: Identifies instructions and data that depend on absolute addresses.
- **Symbol table**: Associates addresses with external labels and lists unresolved references.
- **Debugging info**: Information used for debugging.

Interpreters

An **interpreter**, like a compiler, translates high-level language into low-level machine language.

- The difference lies in the way they read the source code or input.
- A compiler reads the whole source code at once, creates tokens, checks semantics, generates intermediate code, executes the whole program, and may involve many passes.
- An interpreter reads a statement from the input, executes it, then takes the next statement in sequence.
- If an error occurs, an interpreter stops execution and reports it. The interpreter moves on to the next line for execution only after the removal of the error. A compiler reads the whole program even if it encounters several errors.
- Examples: Perl, Python, and Matlab.

Linkers

> Tool that merges the object files produced by separate compilation or assembly and creates a single executable file.

- Linkers are also called link editors.
- A linker tool is used to link all the parts of the program together for execution (executable machine code).

Tasks of a Linker

- Search and locate referenced routines in a program.
- Searches the program to find library routines used by the program, e.g., `printf()`, math routines.

EXTERN STORAGE CLASS Examples

Example 1

PGM1.CPP

```c
#include <stdio.h>
extern int i;
#include "pgm2.cpp"
void show() {
    printf("\n value of i in pgm2.cpp=%d",i);
}
int main() {
    i=10;
    show();
    printf("\n Value of i in pgm1.cpp=%d",i);
    return 0;
}
```

PGM2.CPP

```c
void show(void);
int i;
```

Example 2

file.h

```c
extern int x = 10,y = 20;
```

demo.c

```c
#include "file.h"
#include <stdio.h>

void main() {
    int add;
    printf("x = %d \n",x);
    printf("y = %d \n",y);
    add=x+y;
    printf("Add = %d \n",add);
}
```

Loaders

Loader is the program of the operating system which loads the executable from the disk into the primary memory (RAM) for execution.

- It allocates the memory space to the executable module in main memory and then transfers control to the beginning instruction of the program.
- It places programs into memory and prepares them for execution.
- Loading a program involves reading the contents of the executable file containing the program instructions into memory and then carrying out other required preparatory tasks to prepare the executable for running.
- Once loading is complete, the operating system starts the program by passing control to the loaded program code.

Relocation

Relocation is the process of assigning load addresses for position-dependent code and data of a program and adjusting the code and data to reflect the assigned addresses.

- The available memory is generally shared among a number of processes in a multiprogramming system.
- It is not always possible to know in advance which other programs will be resident in main memory at the time of execution of a program.
- Swapping the active processes in and out of the main memory enables the operating system to have a larger pool of ready-to-execute processes.
- When a program gets swapped out to a disk memory, it is not always possible that when it is swapped back into main memory it occupies the previous memory location.

Sample Questions

Question 1

In a resident-OS computer, which of the following system software must reside in the main memory under all situations?

(A) Assembler

(B) Linker

(C) Loader

(D) Compiler

Answer: (C)

Explanation: The loader is the part of an operating system responsible for loading programs and libraries, placing them into memory, and preparing them for execution.

Question 2

A linker program:

a. places the program in the memory for the purpose of execution.

b. relocates the program to execute from the specific memory area allocated to it.

c. links the program with other programs needed for its execution.

d. interfaces the program with the entities generating its input data.

Answer: c)

Operating Systems

> A program that acts as an intermediary between a user of a computer and the computer hardware.

- Provides an environment in which a user can execute programs in a convenient and efficient manner.

Operating System Goals

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use.
- Use the computer hardware in an efficient manner.

Four Components of a Computer System

1. Hardware
2. Operating system
3. System & Application programs
4. Users

Computer System Structure

- **Hardware**: Provides basic computing resources (CPU, memory, I/O devices).
- **Operating system**: Controls and coordinates the use of hardware among various applications and users.
- **Application programs**: Define the ways in which the system resources are used to solve the computing problems of the users (word processors, web browsers, database systems, video games).
- **Users**: People, machines, other computers.

The OS controls and coordinates the use of hardware among the various application programs for the various users.

Evolution of Operating Systems

- Mainframe System
- Batch Systems
- Multiprogrammed Systems
- Time Sharing Systems
- Desktop Systems
- Multiprocessor Systems
- Distributed Systems
- Real Systems
- HandHeld Systems

Mainframe System

- First computers to tackle commercial and scientific applications.
- Growth from Batch systems to Multi-programmed to Time-sharing (Multitasking system).

Batch System

- Early computers were physically enormous machines run from a console.
- Input devices: Card readers and Tape drivers.
- Output devices: Line printers, Tape drivers, Card Punches.

    A **tape drive** is a data storage device that reads and writes data on a magnetic tape.

    - Magnetic tape data storage is typically used for offline, archival data storage. Tape media generally has a favorable unit cost and a long archival stability. Magnetic tapes are typically placed in plastic covers referred to as cassettes.
    - A tape drive provides sequential access storage, unlike a hard disk drive, which provides direct access storage.
    - A disk drive can move to any position on the disk in a few milliseconds, but a tape drive must physically wind tape between reels to read any one particular piece of data. As a result, tape drives have very large average access times.
    - However, tape drives can stream data very quickly off a tape when the required position has been reached.

    A **punched card** is a piece of stiff paper that can be used to contain digital data represented by the presence or absence of holes in predefined positions. * Punched cards were widely used through much of the 20th century for data input, output, and storage. * Many early digital computers used punched cards for input of both computer programs and data.

In a batch system:

- Users did not interact with the computer systems.
- Users prepared a job which consisted of the programs, data, and control information and submitted it to the computer operator.
- The job was usually in the form of punch cards.
- At some later time, the output appeared.
- The output consisted of the result of the program, as well as a dump of the final memory and register contents for debugging.
- To speed up processing, operators batched together jobs with similar needs and ran them throughout the computer as a group.

Process:

1. Programmers would leave their programs with the operator.
2. The operator would:
    1. Sort programs into batches with similar requirements.
    2. Run each batch as the computer became available.
3. The output would be sent back to the appropriate programmer.

Multiprogramming

> Multiprogramming increases CPU utilization by organizing jobs (code and data) so that the CPU always has one to execute.

- A subset of total jobs in system is kept in memory - job scheduling.
- One job is selected and run via job scheduling.
- When it has to wait for some task, such as I/O, the OS switches to another job.
- In a non-multiprogrammed system, the CPU would sit idle.

In a **non-multi programmed system**:

- As soon as one job leaves the CPU and goes for an I/O task, the CPU becomes idle.
- The CPU keeps waiting and waiting until this job (which was executing earlier) comes back and resumes its execution with the CPU.
- CPU remains free for all this while.
- CPU remains idle for a very long period of time.
- Other jobs which are waiting to be executed might not get a chance to execute because the CPU is still allocated to the earlier job.

In a **multi-programmed system**:

- As soon as one job goes for an I/O task, the Operating System interrupts that job, chooses another job from the job pool, gives the CPU to this new job, and starts its execution.
- The previous job keeps doing its I/O operation while this new job does CPU-bound tasks.
- If the second job also goes for an I/O task, the CPU chooses a third job and starts executing it.

Drawbacks

Multiprogrammed, batched systems provided an environment where the various system resources were utilized effectively, but they did not provide for user interaction with the computer system.

Timesharing (Multitasking)

> Timesharing (multitasking) is a logical extension of multiprogramming.

- The CPU switches jobs so frequently that users can interact with each job while it is running, creating interactive computing.
- The CPU executes multiple jobs by switching among them, typically using a small time quantum. The switches occur so frequently that the users can interact with each program while it is running.
- An interactive computer system provides direct communication between the user and the system. Response time should be short, < 1 second.
- Each user has at least one program executing in memory - process.
- If several jobs are ready to run at the same time, scheduling swapping is needed.
- Virtual memory allows execution of processes not completely in memory.

To obtain a reasonable response time, jobs may have to be swapped in and out of main memory to the disk. Virtual memory is used, which allows the execution of a job that may not be completely in memory. Programs can be longer than physical memory.

In a time-sharing system, each process is assigned some specific quantum of time for which a process is meant to execute.

Example:

Say there are 4 processes P1, P2, P3, P4 ready to execute with a time quantum of 5 nanoseconds (5 ns). As one process begins execution (say P2), it executes for that quantum of time (5 ns). After 5 ns, the CPU starts the execution of the other process (say P3) for the specified quantum of time.

The CPU makes the processes share time slices between them and execute accordingly. As soon as the time quantum of one process expires, another process begins its execution.

The context switch is so fast that the user can interact with each program separately while it is running. This way, the user is given the illusion that multiple processes/tasks are executing simultaneously.

For multitasking to take place, firstly there should be multiprogramming (i.e., the presence of multiple programs ready for execution) and secondly the concept of time-sharing.

Key Differences

Feature	Multiprogramming	Multitasking (Timesharing)
Interaction	No user interaction	Users can interact with each job while it is running
CPU Utilization	Increases CPU utilization	Optimizes for responsiveness
Complexity	Less complex	More complex
Preemption	Non-Preemptive	Preemptive

Sample Question

What is the name of the technique in which the operating system of a computer executes several programs concurrently by switching back and forth between them?

(A) Partitioning

(B) Multi-tasking

(C) Windowing

(D) Paging

Answer: (B)

Explanation: In a multitasking system, a computer executes several programs simultaneously by switching them back and forth to increase the user interactivity.

Multiprocessor Systems

Also called Parallel Systems or Tightly Coupled Systems.

More than one processor in close communication, sharing the computer bus, the clock, and sometimes memory and peripheral devices.

Advantages

- **Increased Throughput**: More work done; speedup ratio with N processors is not N, it is less than N because overhead is incurred in keeping all parts working correctly and contention for shared resources lowers the expected gains.
- **Economy of scale**: Saves money as they share peripherals, mass storage, power supplies.
- **Increased Reliability**: If functions can be distributed properly among several processors, then the failure of one processor will not halt the system, only slow it down.

Types

- **Symmetric Multiprocessor (SMP)**: Each processor runs an identical copy of the OS, and these copies communicate with one another as needed.
- **Asymmetric Multiprocessor (ASMP)**: Each processor is assigned a specific task. A master processor controls the system. The other processors either look to the master for instructions or have predefined tasks. The master allocates the work to the slave processors.

Asymmetric Multiprocessing (ASMP)

- The operating system typically sets aside one or more processors for its exclusive use. The remainder of the processors run user applications.
- In the ASMP model, if the processor that fails is an operating system processor, the whole computer can go down.

Symmetric Multiprocessing (SMP)

Symmetric multiprocessing (SMP) technology is used to get higher levels of performance.

- In symmetric multiprocessing, any processor can run any type of thread. The processors communicate with each other through shared memory.
- SMP systems provide better load-balancing and fault tolerance.
- Because the operating system threads can run on any processor, the chance of hitting a CPU bottleneck is greatly reduced.
- All processors are allowed to run a mixture of application and operating system code. A processor failure in the SMP model only reduces the computing capacity of the system.

Key Differences

Feature	Asymmetric Multiprocessing (ASMP)	Symmetric Multiprocessing (SMP)
Processor Roles	Each processor has a specific task	Any processor can run any type of thread
OS Distribution	One or more processors for OS exclusive use	Each processor runs an identical copy of OS
Failure Impact	Failure of OS processor can halt the system	Processor failure reduces computing capacity
Load Balancing	Less efficient	Better load balancing
Fault Tolerance	Less fault-tolerant	More fault-tolerant

Distributed Systems

Collection of processors that do not share memory or a clock. Each processor has its own local memory.

- The processors communicate with one another through various communication lines such as high-speed buses or telephone lines.
- Collection of separate, possibly heterogeneous, systems networked together.
- A network is a communications path, TCP/IP is the most common network protocol.
- Most OS support TCP/IP including the Windows and UNIX OS.

Networks

Networks are typecast based on the distances between their nodes:

- **Personal Area Network (PAN)**: Short distance of several feet like Bluetooth Devices.
- **Local Area Network (LAN)**: Within a room, a floor, or a building.
- **Metropolitan Area Network (MAN)**: Links buildings within a city.
- **Wide Area Network (WAN)**: Exists between buildings, cities, or countries.

Types of Distributed Systems

- Client-Server Systems
- Peer-to-Peer Systems

Client-Server Systems

> Centralized system act as server systems to satisfy requests generated by client systems.

- The user interface is handled by smart PCs.
- Server systems can be categorized as:
  - **Compute-server system**: Provides an interface to the client to request services (i.e., database), perform actions, and send back results to the client.
  - **File-server system**: Provides a file system interface for clients to store and retrieve files (i.e., create, update, read, and delete files).

Peer-to-Peer (P2P) Systems

> Another model of distributed system P2P does not distinguish clients and servers. All nodes are considered peers.

- A communications model in which each party has the same capabilities and all play equal roles.
- Each node may act as a client, server, or both.
- No peer has a global view of the entire system.
- Peers exchange resources with one another directly.
- Either party can initiate a communication session.
- P2P Networks are autonomous and self-organized with free participation from peers.
- A P2P computing system is formed with peer hosts that are fully distributed without using a centralized server.
- Nodes must join the P2P network, register their services with a central lookup service on the network, or broadcast a request for service and respond to requests for service via a discovery protocol.

P2P Networks are widely used in Messaging, Online Chatting, Video streaming, and Social networking.

BitTorrent

- BitTorrent is one of the most common protocols for transferring large files, such as digital video files containing TV shows or video clips or digital audio files containing songs.

To send or receive files, a person uses a BitTorrent client on their Internet-connected computer. A BitTorrent client is a computer program that implements the BitTorrent protocol.

Popular clients include Torrent, Xunlei, Transmission, qBittorrent, Vuze, Deluge, BitComet, and Tixati.

BitTorrent

The basic idea behind BitTorrent is to split a file into parts and send one part to each peer. Peers can then download the parts they are missing from each other, which reduces both the download time and the load on the server. The BitTorrent protocol is more sophisticated than this simple example, but this illustrates the basic idea. If one peer was downloading from the server, the time for the download to finish will be two times the time.

Real-Time Systems

> Real-time systems are special-purpose operating systems used when rigid time requirements have been placed on processing. They are often used as control devices in dedicated applications.

Real-time operating systems (RTOS) are used in environments where a large number of events, mostly external to the computer system, must be accepted and processed in a short time or within certain deadlines.

Examples of Real-Time Operating Systems

- Satellite launch systems
- Railway signaling systems
- Airline traffic control systems
- Command Control Systems
- Industrial control
- Telephone switching equipment
- Flight control
- Real-time simulations

A real-time system functions correctly only if it returns the correct result within its time constraints. This is in contrast to time-sharing systems, where it is desirable but not mandatory to respond quickly, and batch systems, which may have no time constraints at all.

Types of Real-Time Operating Systems

There are two types of real-time operating systems:

- Hard Real-Time Systems
- Soft Real-Time Systems

Hard Real-Time Systems vs. Soft Real-Time Systems

Feature	Hard Real-Time Systems	Soft Real-Time Systems
Time Constraint	Very strict time constraints; even the shortest possible delay is not acceptable.	Less strict time constraints.
Deadline	Must never miss its deadline.	Can miss its deadline occasionally with some acceptably low probability.
Consequences of Missing Deadline	May have disastrous consequences (e.g., saving life, automatic parachutes, airbags).	No disastrous consequences.
Usefulness of Result	The usefulness of the result decreases abruptly if tardiness increases.	The usefulness of the result decreases gradually with an increase in tardiness.
Examples	Satellite launch systems, railway signaling systems, airline traffic control systems, command control systems.	Telephone switches, multimedia transmission and reception, DVD players, web browsing, gaming-computer games.
Jitter	Hard real-time OS has less jitter than soft real-time OS. Jitter is the variation/displacement between the signals or data being sent. Hard real-time OS require minimized jitter for sensitive systems.	Soft real-time OS do not require strict timing constraints and a bit delay is permissible.

> Tardiness refers to how late a real-time system completes its task with respect to its deadline.

Healthcare and Patient Monitoring

Real-time systems are critical in healthcare, where the speed of data processing can mean the difference between life and death. They ensure that data from patient monitoring systems, such as heart rate monitors, is available to clinicians when and where they need it to keep patients safe and healthy.

SPOOLing

SPOOL is an acronym for simultaneous peripheral operations on-line.

It involves using a large buffer from a hard disk to store data through I/O, as I/O operations are slow compared to CPU speeds. Spooling is a specialized form of multi-programming for copying data between different devices, typically used to mediate between a computer application and a slow peripheral like a printer. A dedicated program, the spooler, maintains an orderly sequence of jobs for the peripheral and feeds it data at its own rate.

Clustered Systems

Clustered systems are like multiprocessor systems but consist of multiple systems working together, usually sharing storage via a storage-area network (SAN).

Key Aspects of Clustered Systems

- **High Availability:** Provides a high-availability service that survives failures.
- **High-Performance Computing (HPC):** Used for high-performance computing, requiring applications to be written to use parallelization.
- **Distributed Lock Manager (DLM):** Some clusters have a distributed lock manager to avoid conflicting operations.

Cluster for Massive Parallelism

A collection of interconnected stand-alone computers that can work together collectively and cooperatively as a single integrated computing resource pool. This setup improves speed and/or reliability compared to a single computer, while being more cost-effective than a single computer of comparable speed or reliability.

Cluster Architecture

Scalable clusters can be built through hierarchical construction using a SAN, LAN, or WAN, allowing for an increasing number of nodes. The cluster is connected to the internet via a virtual private network (VPN) gateway.

Single System Image (SSI)

> An ideal cluster merges multiple system images into a single system image (SSI).

Cluster designers aim for a cluster operating system to support SSI at various levels, including the sharing of CPUs, memory, and I/O across all cluster nodes. SSI is an illusion created by software or hardware that presents a collection of resources as one integrated, powerful resource, making the cluster appear like a single machine to the user.

SSI Example

Suppose a workstation has:

- A 300 Mflops/second processor
- 512 MB of memory
- 4 GB disk
- Support for 50 active users and 1,000 processes

By clustering 100 such workstations, the goal is to achieve a single system (a megastation) that has:

- 30 Gflops/second processor
- 50 GB of memory
- 400 GB disk
- Support for 5,000 active users and 100,000 processes

SSI techniques are aimed at achieving this goal.

Asymmetric vs. Symmetric Clustering

- **Asymmetric Clustering:** One machine is in hot-standby mode, while the primary node runs applications. The hot standby host monitors the active server and becomes the active server if the primary server fails.
- **Symmetric Clustering:** Multiple nodes are running applications and monitoring each other. This is more efficient as it uses all available hardware.

OS Objectives/Operations

- Process Management
- Memory Management
- Storage Management
- Mass-Storage Management
- Protection System

Process Management Activities

The operating system is responsible for:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

Memory Management

> To execute a program, all (or part) of the instructions must be in memory, and all (or part) of the data that is needed by the program must be in memory.

Memory management determines what is in memory and when, optimizing CPU utilization and computer response to users.

Memory Management Activities

- Keeping track of which parts of memory are currently being used and by whom
- Deciding which processes (or parts thereof) and data to move into and out of memory
- Allocating and deallocating memory space as needed

Storage Management

The OS provides a uniform, logical view of information storage, abstracting physical properties to a logical storage unit (file). Each medium is controlled by a device (e.g., disk drive, tape drive). Varying properties include access speed, capacity, data-transfer rate, and access method (sequential or random).

File System Storage Management

- Files are usually organized into directories.
- Access control on most systems determines who can access what.

OS Activities Include

- Creating and deleting files and directories
- Primitives to manipulate files and directories
- Mapping files onto secondary storage
- Backup files onto stable (non-volatile) storage media

Mass-Storage Management

Disks are usually used to store data that does not fit in main memory or data that must be kept for a long period. The entire speed of computer operation hinges on the disk subsystem and its algorithms.

OS Activities Include

- Free-space management
- Storage allocation
- Disk scheduling

Some storage need not be fast. Tertiary storage includes optical storage and magnetic tape, which must be managed by the OS or applications. This varies between WORM (write-once, read-many-times) and RW (read-write).

Protection and Security

Protection is any mechanism for controlling access of processes or users to resources defined by the OS.

Security is the defense of the system against internal and external attacks, including denial-of-service, worms, viruses, identity theft, and theft of service.

Systems generally first distinguish among users to determine who can do what. User identities (user IDs, security IDs) include a name and associated number, one per user. The user ID is then associated with all files and processes of that user to determine access control. Group identifiers (group IDs) allow a set of users to be defined, and controls are managed, then also associated with each process and file. Privilege escalation allows a user to change to an effective ID with more rights.

It's a mechanism that ensures that files, memory segments, CPU, and other resources can be operated on only by those processes that have gained proper authorization from the OS.

Operating System Functions/Services

One set of operating-system services provides functionalities that are helpful to the user:

- **User Interface (UI):** Almost all operating systems have a user interface, varying between Command-Line (CLI), Graphics User Interface (GUI), and Batch.
- **Program Execution:** The system must be able to load a program into memory and run that program, end execution, either normally or abnormally (indicating error).
- **I/O Operations:** A running program may require I/O, which may involve a file or an I/O device.
- **File-System Manipulation:** Programs need to read and write files and directories, create and delete them, search them, list file information, and manage permissions.
- **Communications:** Processes may exchange information, on the same computer or between computers over a network. Communications may be via shared memory or through message passing (packets moved by the OS).
- **Error Detection:** The OS needs to be constantly aware of possible errors, which may occur in the CPU and memory hardware, in I/O devices, or in user programs. For each type of error, the OS should take the appropriate action to ensure correct and consistent computing, including debugging facilities.

Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing:

- **Resource Allocation:** When multiple users or multiple jobs are running concurrently, resources must be allocated to each of them. Many types of resources include CPU cycles, main memory, file storage, and I/O devices.
- **Accounting:** To keep track of which users use how much and what kinds of computer resources. This record can be used for billing purposes or for simply accumulating usage statistics.
- **Protection and Security:** Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders is also important, requiring user authentication and extending to defending external I/O devices from invalid access attempts, recording all such connections for detection of break-ins.

Operating System Structure

General-purpose OS is a very large program. There are various ways to structure it:

- Simple structure (MS-DOS)
- More complex (UNIX)
- Layered abstraction
- Microkernel (Mach)

Simple Structure -- MS-DOS

MS-DOS was written to provide the most functionality in the least space. It was designed and implemented by a few people who had no idea that it would become so popular.

- It was not divided into modules carefully due to the limited hardware it was built on.
- Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated.
- MS-DOS was gradually superseded by OS offering a graphical user interface (GUI), in various generations of the graphical Microsoft Windows operating system.
- MS-DOS went through eight versions, until development ceased in 2000.

Non-Simple Structure -- UNIX

Limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts:

- Systems programs
- The kernel

Traditional UNIX System Structure

The kernel consists of everything below the system-call interface and above the physical hardware, further separated into a series of interfaces and device drivers which were added and expanded over the years as UNIX evolved. The kernel provides the file system, CPU scheduling, memory management, and other functions; a large number of functions for one level. This makes UNIX difficult to enhance, as changes in one level could adversely affect other areas.

> **UNIX** is a family of multitasking, multiuser computer operating systems that derive from the original AT&T Unix, developed starting in the 1970s at the Bell Labs research center by Ken Thompson, Dennis Ritchie, and others.
>
> **Multitasking** is the concurrent execution of multiple tasks (also known as processes) over a certain period of time. New tasks can interrupt already started ones before they finish.
>
> **Multiuser** operating systems support simultaneous access by multiple users.

The most popular varieties of UNIX are Sun Solaris, GNU/Linux, and MacOS X. Linux is a clone of Unix. It has several features similar to Unix, still have some key differences.

> **Linux** is a family of open-source Unix-like operating systems based on the Linux kernel.

Layered Approach

The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0) is the hardware, and the highest layer (layer N) is the user interface.

- An OS layer, say M, consists of data structures and a set of routines that can be invoked by higher layers.
- Layer M can invoke operations on lower layers.
- With modularity, layers are selected such that each uses functions and services of only lower-level layers.
- It simplifies debugging and system verification.
- The 1st layer can be debugged without any concern for the rest of the system.
- Once the 1st layer is debugged, its correct functioning can be assumed while the second layer is being debugged, and so on.
- If an error arises during debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged.

Microkernel System Structure

Moves as much from the kernel into userspace. Mach is an example of a microkernel. Mac OS X kernel (Darwin) is partly based on Mach. Communication takes place between user modules using message passing.

How?

Remove all non-essential components from the kernel and implement them as system and user-level programs. This results in a smaller kernel.

Microkernels Provide

Minimal process and memory management, and communication facilities.

Main Function

Provide a communication facility between a client program and various services that are also running in user space. Communication is provided by message passing.

When/Where is it Used?

As the UNIX OS expanded, the kernel became large and difficult to manage. In the mid-1980s, researchers at Carnegie Mellon University developed an OS called Mach that modularizes the kernel using the Microkernel approach.

Benefits

- Easier to extend a microkernel. All new services are added to user space, with no modification of the kernel.
- Easier to port the operating system to new architectures. As no modifications to the kernel are needed, changes tend to be fewer, resulting in a smaller kernel.
- More reliable (less code is running in kernel mode) and more secure. More services running as user rather than kernel, so if a service fails, the rest of the operating system remains untouched.

Detriments

Performance overhead of user space to kernel space communication.

Linux Kernel

> The **kernel** is a busy personal assistant that manages all system resources and that interacts directly with the computer hardware.

The Linux kernel is the main component of a Linux operating system (OS) and is the core interface between a computer's hardware and its processes. It communicates between the two, managing resources as efficiently as possible.

The kernel is named so because, like a seed inside a hard shell, it exists within the OS and controls all the major functions of the hardware.

The Linux kernel forms the core of the Linux project, but other components make up the complete Linux operating system. The Linux kernel is composed entirely of code written from scratch specifically for the Linux project, whereas the Linux system includes a multitude of components, some written from scratch, others borrowed from other development projects, and others created in collaboration with other teams.

If implemented properly, the kernel is invisible to the user, working in its own little world known as kernel space, where it allocates memory and keeps track of where everything is stored. What the user sees, like web browsers and files, are known as the user space. These applications interact with the kernel through a system call interface (SCI).

How the Kernel Fits Within the OS

A Linux machine has 3 layers:

- The hardware
- The Linux kernel: The core of the OS. It's software residing in memory that tells the CPU what to do.
- User processes: These are the running programs that the kernel manages, collectively making up user space.

The kernel also allows these processes and servers to communicate with each other (known as inter-process communication, or IPC).

Core Subsystems of the Linux Kernel

- The Process Scheduler/Process management
- The Memory Management Unit (MMU)
- The Virtual File System (VFS) /File management
- The Networking Unit
- Inter-Process Communication Unit
- Device management/ I/O management

Although it is often mistaken that Linus Torvalds has developed the Linux OS, he is only responsible for the development of the Linux kernel.

> Complete Linux system = Kernel + GNU system utilities and libraries + other management scripts + installation scripts.

Shell

> A **shell** is a special user program that provides an interface for a user to use operating system services.

It accepts human-readable commands from the user and converts them into something which the kernel can understand. It is a command language interpreter that executes commands read from input devices such as keyboards or from files. The shell gets started when the user logs in or starts the terminal.

It is named a shell because it is the outermost layer around the operating system.

Shell Classifications

Shell is broadly classified into two categories:

- Command Line Shell
- Graphical Shell

Command Line Shell

The shell can be accessed by the user using a command-line interface. A special program called Terminal in Linux/macOS or Command Prompt in Windows OS is provided to type in the human-readable commands such as `cat`, `ls`, etc. for execution. The result is then displayed on the terminal to the user.

Working with a command-line shell is a bit difficult for beginners because it is hard to memorize so many commands. However, it is very powerful, allowing the user to store commands in a file and execute them together. This way, any repetitive task can be easily automated. These files are usually called Batch files in Windows and Shell Scripts in Linux/macOS systems.

Graphical Shells

Provide means for manipulating programs based on a graphical user interface (GUI) by allowing for operations such as opening, closing, moving, and resizing windows, as well as switching focus between windows. Window OS or Ubuntu OS can be considered as good examples which provide GUI to the user for interacting with the program. The user does not need to type in commands for every action.

CLI vs. GUI Shells

Feature	Command-Line Shells	Graphical Shells
User Burden	Require the user to be familiar with commands and their calling syntax and to understand concepts about the shell-specific scripting language.	Place a low burden on beginners to be familiar with and are easy to use.
Other Information	Since they also come with their own disadvantages, most GUI-enabled operating systems also provide CLI shells.	-

Shell Prompt

> The prompt, $, which is called the **command prompt**, is issued by the shell.

While the prompt is displayed, you can type a command. The shell reads your input after you press Enter. It determines the command you want to be executed by looking at the first word of your input. A word is an unbroken set of characters. Spaces and tabs separate words.

Finding Out Your Shell

When you are provided with a UNIX account, the system administrator chooses a shell for you. To find out which shell was chosen for you, look at your prompt:

- If you have a `$` prompt, you're probably in a Bash, Bourne, or Korn shell.
- If you have a `%` prompt, you're probably in a C shell.

You can also use the `echo` command.

Listing All Shells on Your PC

You can use the `cat` command to list all the shells on your PC. The list of all the shells which are currently installed in our Linux system is stored in the `shells` file which is present in the `/etc` folder of the system. It has read-only access by default and is modified automatically whenever we install a new shell in our system.

> The `/etc` directory contains configuration files, which can generally be edited by hand in a text editor. Note that the `/etc/` directory contains system-wide configuration files. User-specific configuration files are located in each user's home directory.

The `cat` command displays the various installed shells along with their installation paths.

Changing the Shell

Use the `chsh` utility. `chsh` is the utility to change a user's login shell. `chsh` provides the `-s` option to change the user's shell.

There are several shells available for Linux systems, and each shell does the same job. Okay, here's your study guide on shells, shell scripting, the boot process, system calls, device drivers, OS design, and multiprocessing.

Shell Types in UNIX

In UNIX, a **shell** serves as an interface between the user and the operating system kernel. It interprets commands entered by the user and translates them into actions that the OS can perform. There are two major types of shells:

- The **Bourne shell**: Default prompt is the `$` character.
- The **C shell**: Default prompt is the `%` character.

The Bourne Shell

- The original UNIX shell, written in the mid-1970s by Stephen R. Bourne at AT&T Bell Labs.
- Often referred to as "the shell" since it was the first to appear on UNIX systems.
- Denoted as `sh`.
- Faster and often preferred for scripting.
- Lacks interactive features like recalling previous commands and built-in arithmetic/logical expression handling.
- Default shell for Solaris OS.

CSH (C Shell)

- Denoted as `csh`.
- Created by Bill Joy at the University of California, Berkeley.
- Incorporates features like aliases and command history.
- Includes programming features like built-in arithmetic and C-like expression syntax, making it similar to the C programming language.

Shell Subcategories

Bourne Shell:

- Bourne shell (`sh`)
- Korn shell (`ksh`)
- Bourne Again shell (`bash`)
- POSIX shell

C Shell:

- C shell (`csh`)
- TENEX/TOPS C shell (`tcsh`)

The Korn Shell

- Denoted as `ksh`.
- Written by David Korn at AT&T Bell Labs.
- A superset of the Bourne shell, supporting everything in the Bourne shell.
- Has interactive features, including built-in arithmetic, C-like arrays, functions, and string manipulation facilities.
- Faster than C shell.
- Compatible with scripts written for C shell.
- Served as the base for the POSIX Shell standard specification.

BASH (Bourne Again Shell)

- Stands for **Bourne Again Shell**. A pun on the name of the Bourne shell that it replaces.
- Command syntax is a superset of the Bourne shell command syntax.
- Includes ideas from the KornShell (`ksh`) and the C shell (`csh`), such as command line editing, command history (`history` command), directory stack, and POSIX command substitution syntax.
- Denoted as `bash`.
- Compatible with the Bourne shell.
- Most widely used shell in Linux systems.
- Default login shell in Linux systems and macOS. Can also be installed on Windows OS.

GNU

> GNU is an operating system and an extensive collection of utility programs, which are all free software. It is also the project within which the free software concept originated. GNU is a recursive acronym for "GNU's Not Unix!" chosen because GNU's design is Unix-like but differs from Unix by being free software and containing no Unix code.

Shell Scripting

Shells are typically interactive, accepting commands from users and executing them.

Shell Script:

> A file containing a series of commands that can be executed by the shell to avoid repetitive typing of commands in the terminal. Shell scripts are similar to batch files in MS-DOS and are saved with a `.sh` file extension (e.g., `myscript.sh`).

A shell script includes:

- Shell **keywords** (e.g., `if`, `else`, `break`).
- Shell **commands** (e.g., `cd`, `ls`, `echo`, `pwd`, `touch`).
- **Functions**.
- **Control flow** (e.g., `if..then..else`, `case`, and shell loops).

The Shell Definition Line

The shell definition line specifies which shell should be used to interpret the script's commands. For example, `#!/bin/ksh` tells the system to use the Korn Shell.

Shell Script Comments

Comments in a shell script begin with the `#` (pound) character.

Script Permissions

Before running a script, you need to make it executable using the `chmod` command.

- Allow only the user (owner) to execute: `$ chmod u+x script1`
- Allow everyone to execute: `$ chmod a+x script1`

Executing Scripts

After making a script executable, you can run it by giving the path to the script:

- `$ ./script1` (if in the same directory)
- `$ /home/student1/script1` (if in a different directory)
- `$ /bin/ksh /home/student1/script1`
- On some online portals: `$ sh script1.sh`

Sample Program

1. Open a file using `vi hello.sh`.
2. Press `ESC` and type `I` to enter edit mode.
3. Save the file: press `ESC` and type `:w fileName`.
4. Save and quit: press `ESC` and type `:wq` or `:x`.
5. Run the script: `./hello.sh`.
6. If you get a "Permission denied" error, type `chmod u+x hello.sh` and execute the program again.

chmod Command

The `chmod` command is used to change the access permissions of a file. The syntax is:

```
chmod ugo+rwx filename
```

Where:

- `u`: Users
- `g`: Groups
- `o`: Others
- `+`: Adds the permission
- `-`: Removes the permission
- `=`: Overwrites current permissions
- `r`: Read permission
- `w`: Write permission
- `x`: Execute permission

Displaying Output

Two common methods for displaying output:

- `echo "Text Line 1"`
- `print` (more powerful and standardized replacement for `echo`)

Echo Command

Displays a line of text. For example:

```
$ echo $x
Output: 10 (if x=10)
```

Double quotes with `echo` affect how spaces and TAB characters are treated.

Read Command

Reads input from the user.

Defining Variables

Define a variable using the syntax `variable_name=value`. Get the value by adding `$` before the variable name (e.g., `$variable_name`).

Arithmetic Expression

Arithmetic expansion allows the evaluation of an arithmetic expression and substitution of the result. The format is:

```
$((expression))
```

The older format `$[expression]` is deprecated.

Shell Script to Add Two Integers

```
#!/bin/bash
# Calculate the sum of two integers with pre initialized values
# in a shell script
a=10
b=20
sum=$(( $a + $b ))
echo $sum
```

Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo (remainder)
**	Exponentiation (some shells)

Relational Operators

Operator	Description
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

Shell Script Using Decision Constructs

```
Syntax
if [ expression ]
then
    Statement(s) to be executed if expression is true
else
    Statement(s) to be executed if expression is not true
fi
```

Conditional expressions should be inside square braces with spaces around them (e.g., `[ $a == $b ]` is correct, while `[$a==$b]` is incorrect).

Conditional Expressions

- `-lt` is less than, used for condition checking.
- `<` is used for reading input from files.

The statement `[ $num < 10 ]` is syntactic sugar for the command `test $num < 10`.

Shell Script to Check if a Number is Even or Odd

`expr` is an external program used by the Bourne shell, employed with the help of backticks for command substitution.

Shell Script to Calculate HRA and TA

`expr` is an external program used by the Bourne shell, employed with the help of backticks for command substitution.

Backslash Character

The backslash `\` character is used to escape special characters, preventing them from being interpreted by the shell.

For Loop Syntax

```
condition1 indicates initialization
cond2 indicates condition
cond3 indicates updation
```

System Boot

The boot process is what happens every time you turn on your computer.

1. The CPU initializes itself after the power is turned on, triggered by clock ticks from the system clock.
2. The CPU looks for the system's ROM BIOS. The first instruction is stored in the ROM BIOS and instructs the system to run POST (Power On Self Test) in a predetermined memory address.
3. POST checks the BIOS chip, hardware devices, CMOS RAM, secondary storage devices, and other hardware (mouse, keyboard) to ensure they are working properly.
4. After POST, the BIOS starts the Boot Strap program.

BootStrap

- For most computers, the BootStrap is stored in BIOS ROM.
- A tiny Bootstrap loader program is stored in the ROM, whose only job is to bring in a full bootstrap program from the disk.
- The Bootstrap program is loaded into memory and starts its execution.
- The bootstrap program finds the OS kernel on disk, loads that Kernel into memory, and jumps to an initial address to begin the OS execution.

Master Boot Record (MBR)

The Master Boot Record (MBR) is the information located in the first sector of any hard disk. It holds information about GRUB (or LILO in old systems).

Init

This is the last step of the booting process. It decides the run level by looking at the `/etc/inittab` file. The initial state of the operating system is decided by the run level.

Run levels:

Level	Description
0	System Halt
1	Single user mode
2	Multiuser, without NFS (Network File System)
3	Full multiuser mode
4	Unused
5	Full multiuser mode with network and X display manager
6	Reboot

The system then:

1. Starts up various **daemons** that support networking and other services.
2. The **X server daemon** manages the display, keyboard, and mouse.
3. A Graphical Interface appears, and a login screen is displayed when the X server daemon is started.

   Daemons are processes that run unattended in the background and are available at all times, typically started during system startup and running until the system stops.

Failure During Boot

If the computer cannot boot, a boot failure error occurs. This indicates that the computer is not passing POST or a device has failed.

Cold Boot vs. Warm Boot

- **Cold Boot (Hard Boot)**: Starting a computer that is turned off.
- **Warm Boot**: Restarting a computer once it has been turned on, often used if the computer hangs.

UNIX Permissions

- UNIX provides three types of permissions: **Read, Write, Execute**.
- UNIX provides three sets of permissions: **permission for owner, permission for group, permission for others**.

System Calls

System calls provide the interface between a process and the OS.

- System calls are usually made when a process in user mode requires access to a resource.
- System calls are generally available as assembly language instructions.
- Languages like C, C++, Perl have been defined to replace Assembly language for system programming.

When are System Calls Required?

1. If a file system requires the creation or deletion of files.
2. Reading and writing from files also require a system call.
3. Creation and management of new processes.
4. Network connections also require system calls including sending and receiving packets.
5. Access to a hardware devices such as a printer, scanner etc. requires a system call.

System Call Process

1. Processes execute normally in user mode until a system call interrupts this.
2. The system call is executed on a priority basis in kernel mode.
3. After execution, control returns to user mode.
4. Execution of user processes can be resumed.

System Call Implementation

1. A number is associated with each system call.
2. The system-call interface maintains a table indexed according to these numbers.
3. The system call interface invokes the intended system call in the OS kernel and returns the status and any return values.

API, System Call, OS Relationship

The caller needs to know nothing about how the system call is implemented, just the API and what the OS will do as a result of the call. Most details of the OS interface are hidden from the programmer by the API, managed by the run-time support library.

System Call Parameter Passing

System calls occur in different ways and often require more information than simply the system call identity.

Three general methods to pass parameters to the OS:

1. Pass the parameters in registers.
2. Store parameters in a block or table in memory and pass the address of the block in a register (used by Linux and Solaris).
3. Place parameters onto the stack.

Types of System Calls

Category	Examples
Process control	Create process, terminate process, end, abort, load, execute, get process attributes, set process attributes, wait for time, wait event, signal event, allocate and free memory, dump memory if error, debugger, locks for managing access to shared data between processes
File management	Create file, delete file, open, close file, read, write, reposition, get and set file attributes
Device management	Request device, release device, read, write, reposition, get device attributes, set device attributes, logically attach or detach devices
Information maintenance	Get time or date, set time or date, get system data, set system data, get and set process (process id), file, or device attributes
Communications	Create, delete communication connection, send, receive messages (message passing model), shared-memory model, transfer status information, attach and detach remote devices
Protection	Control access to resources, get and set permissions, allow and deny user access

Layered File System

Device Drivers

> Device drivers manage I/O devices at the I/O control layer and control the physical device.

The I/O control consists of device drivers and interrupt handlers to transfer information between main memory and the disk system. A device driver can be thought of as a translator, converting high-level commands into low-level hardware-specific commands.

Device drivers are software modules that can be plugged into an OS to handle a particular device.

Device Controllers

> The Device Controller works like an interface between a device and a device driver. There is always a device controller and a device driver for each device to communicate with the Operating Systems.

I/O units typically consist of a mechanical component and an electronic component (device controller). The controller may be able to handle multiple devices and performs tasks such as converting a serial bit stream to a block of bytes, performing error correction, and making the device available to main memory.

Operating System Design and Implementation

- Design and Implementation of OS is a complex task.
- Start the design by defining goals and specifications, affected by choice of hardware and system type.

User Goals vs. System Goals

- **User goals**: OS should be convenient to use, easy to learn, reliable, safe, and fast.
- **System goals**: OS should be easy to design, implement, and maintain, flexible, reliable, error-free, and efficient.

Important principle: separate:

- **Policy**: What will be done?
- **Mechanism**: How to do it?

Implementation Languages

- Early OSes in assembly language.
- Then system programming languages like Algol, PL/1.
- Now C, C++.
- A mix of languages is common, with lowest levels in assembly and the main body in C.

OS Design Considerations for Multiprocessor and Multicore Architectures

Symmetric Multiprocessor (SMP) OS

> In an SMP system, the kernel can execute on any processor, and typically each processor does self-scheduling from the pool of available processes or threads. The kernel can be constructed as multiple processes or as multiple threads, allowing portions of the kernel to execute in parallel.

The SMP approach complicates the OS because the OS designer must deal with complexity due to sharing resources and coordinating actions from multiple parts of the OS executing at the same time.

OS Design Considerations for SMP:

1. Simultaneous concurrent processes or threads
2. Scheduling
3. Synchronization
4. Memory management
5. Reliability and fault tolerance

That concludes the study guide!

OS Design Considerations for Symmetric Multiprocessor (SMP) OS

When designing an OS for symmetric multiprocessors, several factors must be considered to ensure proper functionality and performance. These considerations address challenges introduced by having multiple processors executing kernel code simultaneously.

Scheduling

- Any processor can perform scheduling, which complicates enforcing a scheduling policy.
- It is important to avoid corruption of the scheduler data structures.
- If kernel-level multithreading is used, there's an opportunity to schedule multiple threads from the same process simultaneously on multiple processors.

Synchronization

- Effective **synchronization** is crucial with multiple active processes accessing shared address spaces or shared I/O resources.
- Synchronization enforces **mutual exclusion** and **event ordering**.
- **Locks** are a common synchronization mechanism used in multiprocessor operating systems.

Memory Management

- Memory management must handle all issues found in uniprocessor computers.
- The OS needs to exploit available hardware parallelism to achieve the best performance.
- Paging mechanisms on different processors must be coordinated to enforce consistency when multiple processors share a page or segment.
- Page replacement strategies must be decided upon.
- The reuse of physical pages is a significant concern: ensuring a physical page can't be accessed with its old contents before being reused.

Reliability and Fault Tolerance

- The OS should provide **graceful degradation** in the face of processor failure.
- The scheduler and other OS components must recognize the loss of a processor and restructure management tables accordingly.

Additional Notes

- Multiprocessor OS design often involves extensions to solutions used for multiprogramming in uniprocessor systems. Therefore, they are not always treated entirely separately.

OS Design Considerations for Multicore Architectures

Considerations for multicore systems include all design issues discussed for SMP systems, but with additional concerns due to the potential scale of parallelism.

Multicore Scaling

- Multicore vendors offer systems with multiple cores on a single chip.
- With each generation, the number of cores and the amount of shared and dedicated cache memory increases, leading to many-core systems.

Unicore vs. Multicore

A **Unicore processor** has a single core.

A **Multicore processor** has two or more cores.

Multicore Advantages

- Cores in a multicore processor can individually read and execute program instructions simultaneously.
- This increases the speed of execution and supports parallel computing. Examples include Quadcore and Octacore processors.

Parallelism within Applications

Subdivision of Tasks

- Most applications can be subdivided into multiple tasks that can execute in parallel.
- These tasks can be further divided into multiple processes, each potentially with multiple threads.
- The developer must decide how to split the application work into independently executable tasks and determine which pieces can or should be executed asynchronously or in parallel.

OS Support

- Compilers and programming language features primarily support parallel programming.
- The OS can support this by efficiently allocating resources among parallel tasks as defined by the developer.

Virtual Machine Approach

Resource Misuse

- With the increasing number of cores on a chip, attempting to multiprogram individual cores to support multiple applications may misuse resources.

Core Dedication

- Instead, dedicating one or more cores to a particular process and allowing the processor to focus on that process can avoid much of the overhead of task switching and scheduling decisions.

Multicore OS as Hypervisor

- The multicore OS can act as a **hypervisor**, making high-level decisions to allocate cores to applications but doing little resource allocation beyond that.

System Calls

- System calls are mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use.
- Three common APIs include:
    - Win32 API for Windows
    - POSIX API for POSIX-based systems (UNIX, Linux, Mac OS X)
    - Java API for the Java Virtual Machine (JVM)